Université Claude Bernard Lyon 1
Master 2 Intelligence Artificielle · Systèmes multi-agents

TP / Projet JADE

Système multi-agents BDI de réservation de taxis

Installation de JADE, premiers agents et modélisation BDI sous JADE 4.5.0

Réalisé par
Yennipena Jérémie COULIBALY
Antoine SALAZAR

Enseignante : Nadia KABACHI
Novembre 2026

1. Introduction

Dans ce TP, nous avons travaillé sur deux grandes parties, l'une consistait en la prise en main JADE: installation, lancement de la plateforme, découverte des agents système et des outils de supervision, écriture d'un premier agent puis échange de messages ACL. La seconde était d'utiliser la plateforme pour modéliser et simuler une application selon l'architecture BDI (Belief, Desire, Intention).

Parmi les trois sujets proposés, nous avons retenu le premier : un système de réservation de taxis dans une ville simulée, où des clients cherchent un transport et où des taxis proposent leurs services. Chaque agent poursuit ses propres buts (être transporté sans dépasser son budget pour le client, gagner de l'argent sans tomber en panne pour le taxi) et c'est l'interaction entre ces comportements autonomes qui produit le fonctionnement global. Les sections 2 à 4 reprennent les questions du sujet, la section 5 présente la modélisation BDI et la section 6 les scénarios simulés.

2. Installation de JADE sous macOS avec IntelliJ IDEA

Le projet a été développé sur un MacBook Pro sous macOS, avec IntelliJ IDEA et JADE 4.5.0. La démarche reste très proche de celle décrite pour Ubuntu dans le sujet ; seules quelques différences propres au Mac sont signalées plus bas.

2.1 Java

JADE demande au minimum un JDK 8. Nous avons utilisé un JDK 17 installé avec le gestionnaire de paquets Homebrew, ce qui évite de passer par l'installateur d'Oracle.

2.2 Récupération de JADE

L'archive JADE-all-4.5.0.zip a été téléchargée depuis le site officiel (jade.tilab.com) puis décompressée. On obtient les dossiers JADE-bin, JADE-doc, JADE-examples et JADE-src. Les deux fichiers utiles sont jade.jar (dans JADE-bin/lib) et jadeExamples.jar, que nous avons copiés dans un dossier lib/ à la racine du projet pour que celui-ci reste autonome.

2.3 Configuration du projet dans IntelliJ IDEA

1. File > New > Project, type Java, SDK homebrew-17, sans système de build (ni Maven ni Gradle).
2. Créer le dossier lib/ et y placer jade.jar et jadeExamples.jar.
3. File > Project Structure > Modules > Dependencies, bouton + > JARs or Directories, puis sélectionner les deux fichiers du dossier lib/ (portée Compile).
4. Créer les packages sous src/ (agents, bdi, utils) et la classe Main.
5. Run > Edit Configurations > + > Application : classe principale Main, arguments de programme nominal (ou le nom d'un autre scénario), répertoire de travail : la racine du projet.

Pour lancer directement la plateforme ou un agent isolé sans passer par notre classe Main, il suffit de créer une seconde configuration dont la classe principale est jade.Boot et dont les arguments sont par exemple -gui -agents hello:firstAgent.HelloAgent.

2.4 Particularités rencontrées sous macOS

- Dans un terminal, le séparateur de classpath est « : » et non « ; » comme sous Windows : java -cp "lib/*:classes" Main nominal. Nous avons ajouté un petit script run.sh qui compile puis lance un scénario.
- Au premier lancement, macOS demande d'autoriser les connexions entrantes pour Java : JADE ouvre le port 1099 (IMTP) et un port HTTP (7778) pour le transport de messages. Il faut accepter, sinon le conteneur principal ne démarre pas correctement.
- Au démarrage, JADE écrit les fichiers APDescription.txt et MTPs-Main-Container.txt à la racine du projet ; ils décrivent l'adresse de la plateforme et peuvent être supprimés sans risque.

3. Premier lancement et découverte de la plateforme

La plateforme se lance depuis la racine du projet avec la commande suivante :

```
java -cp lib/jade.jar jade.Boot -gui
```

L'option -gui démarre l'agent RMA, qui ouvre l'interface graphique de JADE. Le conteneur principal (Main-Container) contient alors trois agents système :

- **AMS** (Agent Management System) : il gère le cycle de vie des agents, leur attribue un identifiant unique (AID) et assure la supervision globale de la plateforme ;
- **DF** (Directory Facilitator) : c'est l'annuaire de services. Un agent y enregistre les services qu'il offre et les autres peuvent l'interroger par type de service ;
- **RMA** (Remote Monitoring Agent) : l'interface graphique elle-même, qui permet de créer, suspendre, tuer ou migrer des agents et de lancer les outils de mise au point.

Depuis le menu Tools du RMA, nous avons utilisé le **DummyAgent**, qui sert à composer et envoyer à la main un message ACL à n'importe quel agent et à observer les réponses, et surtout le **Sniffer**, qui affiche les messages échangés sous forme de diagramme de séquence. L'**Introspector** permet quant à lui de suivre les comportements actifs d'un agent et sa file de messages. Le Sniffer nous a ensuite servi à vérifier le protocole entre clients et taxis.

3.1 L'exemple bookTrading

L'exemple examples.bookTrading est fourni dans jadeExamples.jar. Il se lance avec un acheteur (le titre recherché est passé en argument) et un ou plusieurs vendeurs :

```
java -cp "lib/jade.jar:lib/jadeExamples.jar" jade.Boot -gui -agents \
  "s1:examples.bookTrading.BookSellerAgent;s2:examples.bookTrading.BookSellerAgent;\
   buyer:examples.bookTrading.BookBuyerAgent"
```

Chaque vendeur ouvre une petite fenêtre Swing qui permet d'ajouter un livre et son prix à son catalogue. Le vendeur s'enregistre au DF sous le type « book-selling ». L'acheteur, toutes les minutes, interroge le DF pour connaître les vendeurs, leur envoie un CFP contenant le titre, recueille les PROPOSE (prix) et les REFUSE (livre absent), envoie un ACCEPT_PROPOSAL au moins cher, puis termine dès qu'il reçoit l'INFORM de confirmation. Si le livre est retiré du catalogue entre-temps, le vendeur répond FAILURE. En observant cet échange au Sniffer, on retrouve exactement la structure du protocole FIPA Contract Net que nous avons ensuite réutilisée pour la réservation de taxis.

4. Premier agent et communication entre agents

4.1 HelloAgent

Le premier agent se limite à une classe qui hérite de jade.core.Agent. La méthode setup() joue le rôle de constructeur (takeDown(), symétrique, est appelée avant la destruction) :

```
package firstAgent;
import jade.core.Agent;

public class HelloAgent extends Agent {
   protected void setup() {
      System.out.println("Bonjour, je suis " + getLocalName());
      System.out.println("Mon AID : " + getAID().getName());
      doDelete();
   }
}
```

Lancé avec -agents hello:firstAgent.HelloAgent, l'agent affiche son nom local et son nom global (de la forme hello@192.168.1.12:1099/JADE), puis se termine. Sans l'appel à doDelete(), il reste visible dans l'arbre du RMA.

4.2 Création, envoi et réception de messages

Un message est une instance d'ACLMessage construite avec un performatif. On lui ajoute un ou plusieurs destinataires (des AID), un contenu et éventuellement un protocole, une ontologie ou un identifiant de conversation, puis on l'envoie avec send(). Côté récepteur, un comportement cyclique lit sa file avec receive() et se met en attente avec block() lorsqu'elle est vide, ce qui évite une attente active. La méthode createReply() prépare une réponse dont le destinataire, l'identifiant de conversation et le protocole sont repris du message reçu ; il ne reste qu'à fixer le performatif et le contenu.

```
ACLMessage inform = new ACLMessage(ACLMessage.INFORM);
inform.addReceiver(new AID("vendeur", AID.ISLOCALNAME));
inform.setProtocol("information");
inform.setContent("contenu");
send(inform);
// côté récepteur, dans un CyclicBehaviour
ACLMessage msg = receive(MessageTemplate.MatchPerformative(ACLMessage.INFORM));
if (msg == null) { block(); return; }
ACLMessage reply = msg.createReply();
reply.setPerformative(ACLMessage.CONFIRM);
reply.setContent("bien reçu");
send(reply);
```

Dans bookTrading, nous avons repris ce mécanisme pour faire dialoguer acheteur et vendeurs avec les performatifs CFP, PROPOSE, REFUSE, ACCEPT_PROPOSAL, INFORM et FAILURE, en filtrant les réponses par conversationId et inReplyTo grâce aux MessageTemplate. Pour le projet, nous avons préféré les classes ContractNetInitiator et ContractNetResponder du package jade.proto, qui gèrent elles-mêmes la corrélation des messages et les délais de réponse.

5. Sujet choisi : réservation de taxis selon l'approche BDI

5.1 Environnement et agents

La ville est une grille de 20 × 15 cases parcourue en distance de Manhattan, comme un quartier en damier. Une station-service se trouve au centre, en (10, 7). Un niveau de trafic compris entre 0 et 9 évolue au cours du temps : plus il est élevé, plus un taxi a de chances de rester bloqué sur sa case à chaque pas. Un taxi consomme une unité de carburant par case parcourue, pour un réservoir de 100.

Le système comprend quatre types d'agents. Les **taxis** et les **clients** sont des agents BDI. La **ville** (EnvironmentAgent) fait apparaître les clients selon le scénario, avec un départ, une destination, un budget et une patience tirés au hasard, et diffuse le niveau de trafic aux taxis toutes les six secondes. L'agent **display** n'intervient pas dans la décision : il interroge périodiquement tous les agents et affiche la carte, un tableau de bord et un journal des événements.

5.2 Architecture BDI commune

Plutôt que d'utiliser une extension BDI existante (Jadex, JaCaMo), nous avons écrit une petite couche BDI au-dessus de JADE dans le package bdi. Elle comprend une base de croyances (BeliefBase, un dictionnaire clé/valeur thread-safe), des désirs (Desire : un type, une priorité et des paramètres) et des intentions (Intention : un désir auquel l'agent s'est engagé). La classe abstraite BDIAgent exécute, grâce à un TickerBehaviour, une version simplifiée de la boucle de Rao et Georgeff :

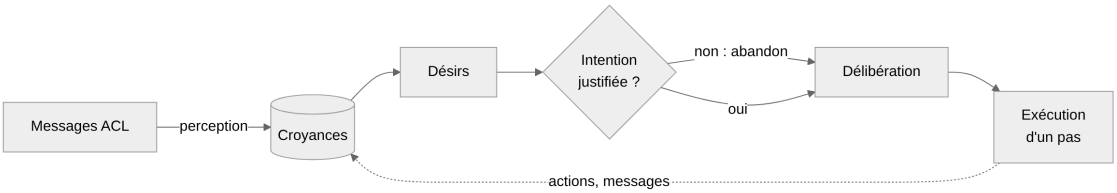


Figure 1 : cycle de raisonnement exécuté à chaque tick par BDIAgent

1. **Perception** : des CyclicBehaviour lisent les messages au fil de l'eau et ne font que réviser les croyances ;
2. **Génération des options** : genererDesirs() produit la liste des désirs compatibles avec les croyances actuelles ;
3. **Reconsidération** : si le désir de l'intention courante ne fait plus partie des options, l'intention est abandonnée ;
4. **Délibération** : le désir de plus forte priorité devient l'intention, en remplaçant l'intention courante s'il est plus prioritaire ;
5. **Exécution** : executer() fait avancer d'un seul pas le plan associé (déplacement d'une case, envoi d'un message…).

La stratégie d'engagement est donc ouverte (open-minded) : l'agent reste fidèle à son intention tant qu'elle est justifiée par ses croyances, mais il peut l'abandonner ou la préempter. Deux désirs sont considérés égaux s'ils ont le même type et les mêmes paramètres, ce qui permet de détecter qu'une intention « aller chercher client3 » n'a plus de raison d'être lorsque client3 a annulé. N'exécuter qu'un pas par tick oblige l'agent à réévaluer sa situation en permanence.

5.3 Agent taxi

Les croyances du taxi sont sa position, son carburant, son solde, le nombre de courses effectuées, le niveau de trafic perçu, sa station d'attache (sa position de départ) et l'état de la course en cours : la phase (réservé, engagé, client à bord, paiement), le client, les points de prise en charge et de destination et le prix convenu. Les désirs sont générés à partir de ces croyances :

Désir	Priorité	Condition de génération et plan associé
ENCAISSER	100	Client déposé : attendre le paiement (délai maximal de 5 s).
TRANSPORTER	90	Client à bord : avancer vers la destination, puis envoyer ARRIVEE avec le prix.
ALLER_CHERCHER	80	Course acceptée : avancer vers le client, puis envoyer PRISE_EN_CHARGE.
FAIRE_LE_PLEIN	70	Libre et carburant < 25 : rejoindre la station et pomper jusqu'au plein.
ATTENDRE_DECISION	50	Offre envoyée : rester réservé, la réservation expire après 5 s.
SE_REPOSITIONNER	20	Libre et à plus de 4 cases de sa station d'attache : y retourner.
ATTENDRE_CLIENT	10	Sinon : intention de fond, jamais atteinte.

Tableau 1 : désirs du taxi

Lorsqu'il reçoit un appel d'offres, le taxi délibère sur sa réponse. Il refuse s'il est déjà occupé ou réservé (« indisponible »), s'il doit faire le plein ou si son carburant ne couvre pas l'approche plus la course avec une marge de trois cases (« carburant »), ou encore si le prix dépasse le budget annoncé (« trop-cher »). Sinon, il calcule son prix, passe en réservation provisoire et répond PROPOSE. Cette réservation évite qu'un même taxi s'engage auprès de deux clients qui appellent au même instant. Le prix suit une formule simple : 4 € de prise en charge, 2 € par case de course et 1 € par case d'approche. Le client paie donc un peu plus cher un taxi éloigné, ce qui favorise naturellement le plus proche.

5.4 Agent client

Le client croit connaître sa position, sa destination, son budget, sa patience (en secondes), le nombre d'appels d'offres déjà lancés, le taxi réservé et le prix convenu. Son objectif final est d'arriver à destination sans dépasser son budget.

Désir	Priorité	Condition de génération et plan associé
PAYER	100	Arrivé à destination : payer le taxi.
QUITTER	90	Payé ou abandon : quitter la simulation après quelques secondes.
ANNULER	60	Taxi réservé mais attente supérieure à la patience : envoyer CANCEL et relancer.
CHERCHER_TAXI	50	Sans taxi et délai de relance écoulé : lancer un appel d'offres.
ATTENDRE_TAXI	40	Taxi en approche : attendre la prise en charge.
ARRIVER	40	À bord : attendre l'arrivée.

Tableau 2 : désirs du client

Parmi les propositions reçues, le client choisit la moins chère puis, à prix égal, celle du taxi le plus proche. S'il ne reçoit aucune offre, il relance trois secondes plus tard, et abandonne au-delà d'un nombre maximal de tentatives fixé par le scénario.

5.5 Protocole d'interaction

La réservation suit le protocole FIPA Contract Net : le client est initiateur, les taxis trouvés au DF sous le type « taxi-service » sont participants. La suite de la course utilise des INFORM distingués par leur ontologie (prise-en-charge, arrivee, paiement) ; l'annulation utilise le performatif CANCEL. Les champs des contenus sont séparés par des points-virgules et décrits dans la classe Ontology.

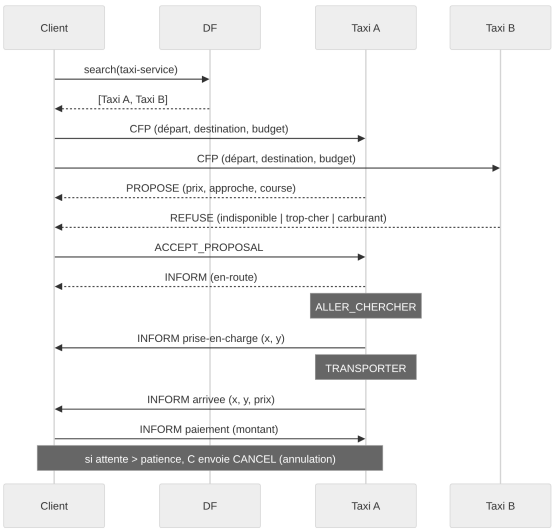


Figure 2 : déroulement d'une réservation complète

5.6 Organisation du code

Classe	Rôle
Main	Crée le conteneur principal (avec le RMA si un écran est disponible), l'affichage, les taxis puis la ville.
bdi.BDIAgent	Boucle BDI générique : options, reconsidération, délibération, exécution.
bdi.BeliefBase / Desire / Intention	Croyances, désirs comparables et intentions.
agents.TaxiAgent	Taxi BDI, participant du Contract Net (ContractNetResponder).
agents.ClientAgent	Client BDI, initiateur du Contract Net (ContractNetInitiator).

Classe	Rôle
agents.EnvironmentAgent	Génération des clients et diffusion du trafic.
agents.GuiDisplayAgent	Carte Swing, tableau de bord et journal.
utils.Scenario / Ontology / Journal	Paramètres des six scénarios, vocabulaire commun et journalisation.

Tableau 3 : principales classes du projet

6. Scénarios et résultats

Six scénarios sont définis dans utils.Scenario ; on les choisit par l'argument de programme de la configuration IntelliJ ou par ./run.sh <scénario>. Dans l'interface, la couleur d'un taxi indique son état (vert libre, jaune réservé, orange en approche, rouge en course, violet en attente de paiement, gris en route vers la station) et la barre sous le taxi son carburant. Les clients sont des disques bleus reliés à leur destination (drapeau). Le tableau de bord affiche l'intention courante de chaque taxi, ce qui permet de suivre la délibération en direct. Une vidéo de chaque scénario est fournie dans le dossier videos/ de l'archive (par exemple videos/embouteillage.mp4) ; elles permettent de voir les déplacements et les changements d'intention, que les captures ci-dessous ne montrent qu'à un instant donné.

- **Nominal** : quatre taxis et une demande modérée. On observe le cycle complet (appel d'offres, réservation, approche, transport, paiement) puis le retour vers la station d'attache. Sur la capture, deux courses ont été payées pour 70 € alors que le trafic est monté à 7/9.
- **Heure de pointe** : deux taxis pour jusqu'à dix clients. Les taxis, presque toujours occupés, répondent « indisponible » ; les clients relancent toutes les trois secondes et certains abandonnent. On compte ici sept clients en recherche.
- **Embouteillage** : trafic entre 6 et 9, patience de 4 à 9 s. Quand client2 annule, le taxi qui venait le chercher efface la course de ses croyances ; le désir ALLER_CHERCHER disparaît et le cycle abandonne l'intention (« taxi2 abandonne l'intention ALLER_CHERCHER client2 »), avant que Taxi2 n'accepte une nouvelle offre du même client.
- **Carburant** : réservoirs entre 18 et 40 %. Taxi1 et Taxi4, sous le seuil, adoptent FAIRE_LE_PLEIN et refusent les courses ; Taxi3, à 30 %, refuse client1 faute de carburant pour l'approche et la course.
- **Budget serré** : budgets à peine supérieurs au prix de la course seule. Les taxis éloignés répondent « trop-cher » : trois taxis restent libres pendant que des clients attendent, et client2 abandonne après six essais.
- **Concurrence** : scénario scripté, sans trafic, donc reproductible. Client1 et client2 obtiennent Taxi1 (18 € = 4 + 2 × 6 + 1 × 2) et Taxi2 (26 €). Client3, arrivé 300 ms plus tard, ne reçoit que des refus, relance, puis obtient Taxi1 pour 39 € dès sa libération. Grâce à la réservation provisoire, aucun taxi n'est attribué à deux clients.

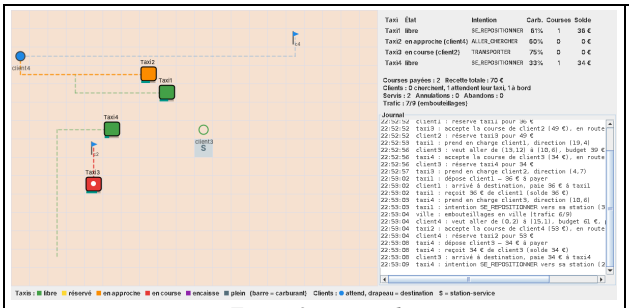


Figure 3 : nominal

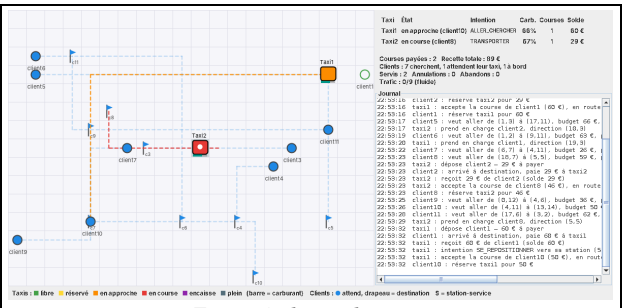


Figure 4 : heure de pointe

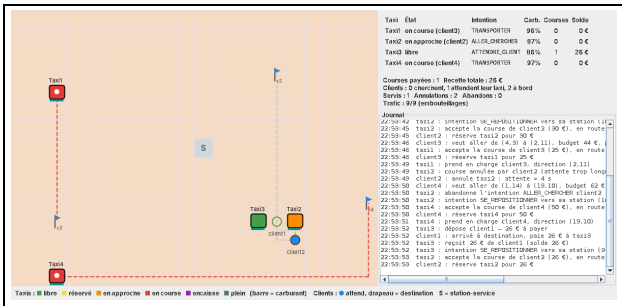


Figure 5 : embouteillage

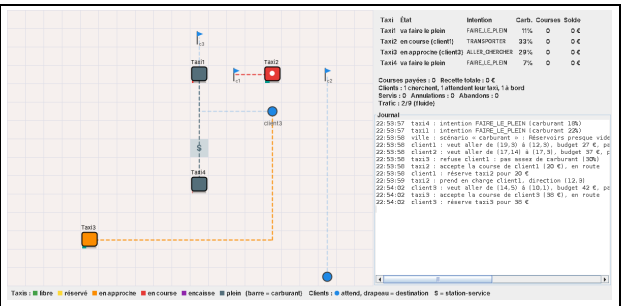


Figure 6 : carburant

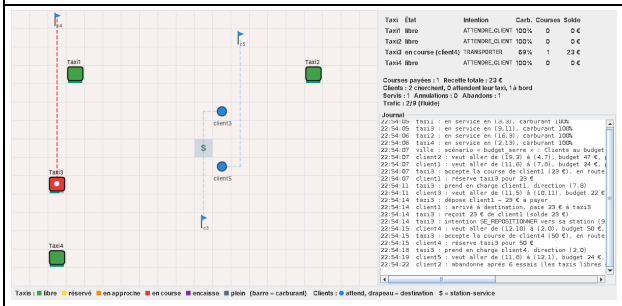


Figure 7 : budget serré

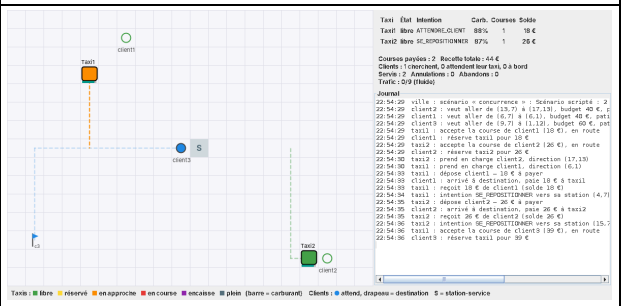


Figure 8 : concurrence

7. Bilan, limites et perspectives

Le système répond au cahier des charges : chaque taxi et chaque client décide seul à partir de ses croyances, et les comportements observés (relances, annulations, refus pour carburant ou budget, abandon d'intention) découlent de la délibération individuelle des agents, sans coordinateur central.

Le modèle reste volontairement simple. Les plans sont codés en dur dans executer() au lieu d'être choisis dans une bibliothèque de plans ; les taxis ne prennent qu'un client à la fois ; le prix n'est pas négocié ; enfin, les délais (expiration des réservations, patience) reposent sur l'horloge réelle, si bien que les simulations aléatoires ne sont pas exactement reproductibles. Plusieurs prolongements seraient intéressants : le covoiturage avec plusieurs clients par taxi, une négociation du prix sur plusieurs tours, des taxis capables d'anticiper la demande en se déplaçant vers les zones les plus actives, ou encore une comparaison avec une implémentation sous Jadex pour évaluer l'apport d'un vrai moteur BDI.